Code of Conduct:

*I **Rakcha A / 192311421** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Rakcha A

SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE:** CSA3711

NAME: Rakcha A **REG NO:** 192311421

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Question 1

An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.

Answer:

1. Scenario analysis. Failures cluster at the minimum and maximum transfer limits, which means the code that compares the amount with the limits (for example, using > instead of >=) is not being exercised with exact boundary inputs. Random testing picks values from a huge range (for example ₹1 to ₹10,00,000), so the probability of hitting exactly the limit values is almost zero. It also has no defined expected result, cannot be repeated reliably, and gives no measure of coverage.

2. Assumed limits. Minimum transfer = ₹100, maximum per transaction = ₹1,00,000, and the account balance is sufficient unless stated otherwise.

3. Equivalence Class Partitioning (ECP).

- Invalid class 1: amount < 100 (including 0 and negative values).
- Valid class: 100 ≤ amount ≤ 1,00,000.
- Invalid class 2: amount > 1,00,000.

- Invalid class 3: non-numeric, blank or special characters.
- Invalid class 4: valid amount but greater than the available balance.

4. Boundary Value Analysis (BVA) and redesigned test cases.

ID	Input (₹)	Class / Boundary	Expected Result
T1	99	Min − 1 (invalid)	Rejected: below minimum limit
T2	100	Minimum (valid)	Transfer successful
T3	101	Min + 1 (valid)	Transfer successful
T4	50,000	Nominal (valid)	Transfer successful
T5	99,999	Max − 1 (valid)	Transfer successful
T6	1,00,000	Maximum (valid)	Transfer successful
T7	1,00,001	Max + 1 (invalid)	Rejected: above maximum limit
T8	0, −500	Invalid class (≤ 0)	Rejected with error message
T9	abc, blank	Invalid format	Rejected with validation message
T10	Amount > balance	Insufficient funds	Rejected; no debit made
T11	Amount = balance	Balance boundary	Successful; balance becomes 0

5. Effectiveness of random testing. Random testing is weak for boundary defects, offers no traceability to requirements and gives no guarantee of completeness. However, it is cheap and can find unexpected input-handling crashes. Decision: **replace it as the primary technique** with ECP + BVA, and **retain it only as a small supplementary activity** (automated fuzzing of the amount field) after the structured tests.

6. Proposed structured and optimized approach.

- Derive ECP classes and BVA values from the requirement (limits, daily limit, balance).
- Add a decision table for combined rules (daily limit exceeded, beneficiary valid, account active/blocked).
- Add state-based and negative tests (session timeout, duplicate submit, network failure during transfer).
- Automate T1–T11 in the regression suite so that boundary tests run on every build.
- Track requirement-to-test traceability and defect leakage to measure improvement.

7. Justification. Every defect-prone boundary is tested with just 11 targeted tests instead of thousands of random ones. This gives maximum defect detection with minimum effort, repeatability, and clear pass/fail criteria. **Conclusion:** ECP + BVA should replace random testing as the core strategy.

Question 2

A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.

Answer:

1. Evaluation of the current approach. The team most likely tested only the "normal" scenario (a valid adult patient with no allergy) and used statement coverage or no formal adequacy criterion at all. Error handling, allergy alerts and dose-capping branches stayed untested, and these are exactly the places where critical defects appear in healthcare software.

2. Conceptual module (dose calculation). Inputs: age, weight, allergy flag.

Node	Statement / Decision	True → / False →
N1	Read age, weight, allergy	→ N2
N2	Decision: weight ≤ 0 OR age < 0?	T → N3 (return error); F → N4
N4	Decision: allergy = true?	T → N5 (raise alert, stop); F → N6
N6	Decision: age < 12?	T → N7 (dose = weight × k); F → N8 (adult fixed dose)
N9	Decision: dose > maxDose?	T → N10 (dose = maxDose); F → N11
N11	Return dose	End

3. Cyclomatic complexity. V(G) = number of decision nodes + 1 = 4 + 1 = **5**, so 5 basis (independent) paths exist. Including the combination of age group and cap, there are 6 complete paths.

4. Coverage achieved and gaps. A single happy-path test (adult, no allergy, no cap) gives about 50–60% statement coverage and only about 25% branch coverage. Missing paths: error path (N3), allergy path (N5), child dose path (N7), capped-dose path (N10). Statement coverage alone is inadequate because a statement can be executed without testing both outcomes of its decision.

5. Improved test set (100% statement, 100% branch, all basis paths).

TC	Age	Weight	Allergy	Path covered	Expected
TC1	30	0	No	N2-T (error)	Error message
TC2	30	70	Yes	N4-T (allergy)	Alert; no dose
TC3	8	25	No	Child, no cap	weight × k
TC4	30	70	No	Adult, no cap	Fixed adult dose
TC5	8	200	No	Child, capped	maxDose
TC6	30	70 (renal)	No	Adult, capped	maxDose

6. Strategy and justification. Adequacy criterion: 100% branch coverage plus all independent paths (path coverage) for the critical dose-calculation logic; statement coverage is only the minimum. Add black-box BVA on weight/age (for example age 11, 12, 13) to complement the white-box tests, and use a coverage tool (JaCoCo/gcov) in the CI pipeline to enforce the criterion.

7. Impact on reliability and risk. Every decision outcome is verified, so wrong-dose and missed-allergy failures are found before release. This reduces patient-safety risk, post-deployment defects and regulatory exposure, and increases confidence in the module.

Question 3

A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.

Answer:

1. Limitations of the current approach. The testers evidently tested each field on its own (single-field ECP/BVA). Defects such as a discount greater than the price, a 100% discount producing a zero or negative total, or a large quantity combined with a large price (overflow) only appear when fields interact. Single-fault testing cannot reveal them.

2. Assumed rules. Price: 1–1,00,000; quantity: 1–100; discount: 0–100%; total = price × quantity × (1 − discount/100).

3. ECP and BVA per field.

Field	Valid class	Invalid classes	Boundary values
Price	1 – 1,00,000	≤ 0; > 1,00,000; non-numeric	0, 1, 2, 99,999, 1,00,000, 1,00,001
Quantity	1 – 100	≤ 0; > 100; decimal; non-numeric	0, 1, 2, 99, 100, 101
Discount %	0 – 100	< 0; > 100; non-numeric	−1, 0, 1, 99, 100, 101

4. Decision table testing (combinations). Conditions: C1 price valid, C2 quantity valid, C3 discount valid, C4 discount = 100%.

Rule	C1	C2	C3	C4	Expected action
R1	Y	Y	Y	N	Compute total correctly
R2	Y	Y	Y	Y	Total = 0 (not negative)
R3	N	Y	Y	–	Price error
R4	Y	N	Y	–	Quantity error
R5	Y	Y	N	–	Discount error
R6	N	N	Y	–	Price and quantity errors
R7	N	Y	N	–	Price and discount errors
R8	Y	N	N	–	Quantity and discount errors
R9	N	N	N	–	All three errors reported

5. Revised strategy.

- Step 1: ECP to define valid and invalid classes for each field.

- Step 2: BVA (robust) on every field to cover limit values.
- Step 3: Decision table rules R1–R9 to cover all logical combinations; each rule gets at least one concrete test using boundary values (for example price = 1,00,000, qty = 100, discount = 100).
- Step 4: Pairwise (all-pairs) testing if more fields (tax, coupon, shipping) are added, to keep the number of tests manageable.
- Step 5: Calculation tests with rounding (for example ₹99.99 × 3 × 7.5% discount).

6. Justification. Exhaustive testing would need about 1,00,000 × 100 × 101 combinations, which is impossible. The ECP + BVA + decision table design covers every class, every boundary and every rule combination with fewer than 40 tests, so defect detection and completeness are much higher than isolated field testing.

Question 4

A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.

Answer:

1. Sample module. for each element x in array a[1..n]: if (x > 0) { if (x is even) evenCount++; else oddCount++; } else negCount++;

2. Gaps in the current approach. The team built an incomplete CFG (for example, they ignored the loop-back edge and the loop-exit edge) and therefore did not list all independent paths. Tests used arrays that always ran the loop at least once with positive even numbers, so other branches and the zero-iteration case were never run.

3. CFG and complexity. Decision nodes: loop condition (D1), x > 0 (D2), x even (D3). V(G) = 3 + 1 = **4** independent paths.

Path	Description	Test data	Expected
P1	Loop not entered	n = 0, a = [ ]	All counts = 0
P2	Loop → x ≤ 0 → negCount	a = [−3]	neg = 1

Path	Description	Test data	Expected
P3	Loop → x > 0 → even	a = [4]	even = 1
P4	Loop → x > 0 → odd	a = [7]	odd = 1

4. Loop and combination tests. Loop boundary tests: 0, 1, 2, n−1, n and n+1 iterations (for example arrays of size 0, 1, 2, max−1, max). One mixed test such as [4, 7, −3, 0, 10] checks that the paths work correctly in sequence. The value 0 is also tested as the boundary of x > 0.

5. Impact of missing independent paths. Every untested path is a possible hidden defect. In this example a defect such as x >= 0 instead of x > 0 or a wrong counter update would remain undetected, and partial path coverage gives false confidence.

6. Justification using coverage criteria. Basis path testing gives 100% branch coverage (and therefore 100% statement coverage) with the minimum of V(G) = 4 tests, and the loop tests add boundary confidence. Full path coverage is impractical with loops, so independent paths plus loop boundaries is the optimal compromise. **Expected outcome:** all branch outcomes verified, every counter checked, and reduced post-release defects.

Question 5

A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.

Answer:

1. Critical analysis. Although both black-box and white-box testing were done, they were probably carried out independently and without a defined link between them. Common weaknesses: (a) no requirement-to-test traceability, (b) the white-box team measured only statement coverage, (c) black-box tests did not cover boundaries and invalid classes, (d) no exit criteria or adequacy thresholds, and (e) gaps found by coverage tools were not fed back into test design.

2. Evaluation of adequacy criteria. Statement coverage is the weakest criterion; it ignores decision outcomes, compound conditions and loops. Without a combined measure (functional

coverage + structural coverage), a module can appear 90% "covered" and still hide defects in interest, tax or rounding logic.

3. Proposed hybrid strategy (example: loan EMI module).

Phase	Activity	Technique / Output
1	Analyse requirements and identify inputs (amount, rate, tenure)	Requirement traceability matrix
2	Design functional tests	ECP + BVA (for example tenure 11, 12, 13 months; amount at limits)
3	Build CFG of the code and compute V(G)	Independent paths list
4	Run phase-2 tests and measure coverage	Coverage tool: statement, branch, path
5	Gap analysis	Add white-box tests for uncovered branches/paths
6	Combination rules	Decision table for rate type, customer category, prepayment
7	Quality gate	Mutation testing and regression automation

4. Exit (adequacy) criteria. 100% of ECP classes and boundaries covered; 100% branch coverage; all basis paths executed; at least 90% of critical-module paths tested; mutation score ≥ 80%; zero open critical defects.

5. Justification. Black-box tests prove the software meets requirements and catch boundary and input errors; white-box tests prove the internal logic is fully exercised. Using coverage results as feedback makes the two techniques complement each other, so missed requirements and missed code are both detected. This prevents defects instead of only finding them, and reduces production failures in a financial setting where errors carry monetary and regulatory risk.

Conclusion: A traceable, metric-driven hybrid strategy with clear exit criteria replaces the disconnected approach and gives better coverage and defect prevention.

Question 6

A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.

Answer:

1. Failure analysis. The incorrect payouts occur exactly at 5, 10 and 15 years, which are the slab boundaries. This strongly indicates an off-by-one comparison error (for example years > 5 instead of years >= 5). The current tests used only typical values such as 3, 7 or 12 years, so the boundaries were never tested.

2. Assumed rules. Service slabs: 0 to < 5 years (Slab A), 5 to < 10 (Slab B), 10 to < 15 (Slab C), ≥ 15 (Slab D). Rating: 1 to 5 (valid).

3. ECP.

- Years: invalid (< 0); Slab A (0–4); Slab B (5–9); Slab C (10–14); Slab D (≥ 15); invalid non-numeric.
- Rating: invalid (< 1); 1, 2, 3, 4, 5 (valid classes); invalid (> 5).

4. BVA test values for years: −1, 0, 1, 4, **5**, 6, 9, **10**, 11, 14, **15**, 16.

5. Redesigned test suite (sample).

TC	Years	Rating	Expected bonus slab
1	−1	3	Rejected (invalid)
2	0 / 1	3	Slab A
3	4	3	Slab A
4	5	3	Slab B (failure point)
5	9	3	Slab B
6	10	3	Slab C (failure point)
7	14	3	Slab C
8	15	3	Slab D (failure point)

TC	Years	Rating	Expected bonus slab
9	16	3	Slab D
10	5, 10, 15	1 and 5	Boundary years with lowest and highest rating
11	10	0 and 6	Rejected (invalid rating)

6. Why ECP and BVA are better than exploratory testing here. (a) Payroll logic is rule-based and deterministic, with defined expected results that can be calculated exactly. (b) Defects concentrate at boundaries, and BVA targets those values systematically, whereas an exploratory tester may not choose exactly 5, 10 and 15. (c) The tests are documented, repeatable and auditable, which is essential for financial compliance. (d) ECP reduces redundant tests while guaranteeing every slab is covered. (e) They can be automated for regression whenever slab rules change. Exploratory testing is still useful as a supplement but cannot guarantee coverage of financial logic.

Question 7

An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.

Answer:

1. Module logic (assumed). Let A = primary sensor failed, B = backup sensor failed, C = temperature out of safe range.

Node	Logic	Flow
N1	Read T, sensor status	→ N2
N2	if (A && (B || C))	T → N3 (trigger ALARM); F → N4
N4	else if (T < Tlow)	T → N5 (heater ON); F → N6
N6	else if (T > Thigh)	T → N7 (cooler ON); F → N8 (idle)
N9	End (merge of N3, N5, N7, N8)	—

2. Gaps in branch coverage. Branch coverage only requires the decision at N2 to be true once and false once. For example, test (A=1, B=1, C=1) gives TRUE and (A=0, B=0, C=0) gives FALSE, so 100% branch coverage is reported. But the combination "primary sensor failed, backup working, temperature out of range" (A=1, B=0, C=1) and the effect of each single condition were never checked. A fault such as (A && B) || C or A && B would pass these two tests, and this is the kind of missed malfunction sequence that occurred in the field.

3. MC/DC strategy. For n = 3 conditions, MC/DC needs a minimum of n + 1 = 4 tests, in which each condition is shown to independently change the decision outcome.

Test	A	B	C	Decision	Independence pair
T1	1	1	0	True	A (T1–T2), B (T1–T3)
T2	0	1	0	False	A
T3	1	0	0	False	B (T1–T3), C (T3–T4)
T4	1	0	1	True	C

Check: T1 and T2 differ only in A and the output changes; T1 and T3 differ only in B; T3 and T4 differ only in C. All three conditions are independently demonstrated. Add normal-range tests for the heater (T < Tlow), cooler (T > Thigh) and idle outputs, plus boundary values at Tlow and Thigh.

4. Impact on safety-critical reliability. MC/DC proves that every individual sensor condition can actually cause the alarm, so masked or incorrect Boolean logic is detected. It is required for the highest-criticality software in standards such as DO-178C (Level A) and is recommended in IEC 61508-type standards. For a laboratory control system it greatly lowers the probability of silent failure and equipment or experiment damage, at a cost of only n + 1 tests instead of 2^n.

Question 8

A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.

Answer:

1. Limitations of isolated testing. Each promo code works correctly when applied alone, so single-input tests pass. The defect appears only when percentage discount, flat discount and tier discount are stacked and their total exceeds the ticket price, giving a negative total. Interaction (combination) faults cannot be found by testing inputs one at a time.

2. Conditions and actions. Conditions: C1 = percentage promo applied, C2 = flat promo applied, C3 = Gold user tier, C4 = total discount ≥ price, C5 = promo code valid. Actions: A1 = charge full price, A2 = apply discounts, A3 = floor total at ₹0 (or block stacking), A4 = reject code.

Rule	C5 valid	C1 %	C2 Flat	C3 Gold	C4 Disc ≥ Price	Expected action
R1	Y	N	N	N	N	A1: full price
R2	Y	Y	N	N	N	A2: % discount
R3	Y	N	Y	N	N	A2: flat discount
R4	Y	N	N	Y	N	A2: tier discount
R5	Y	Y	Y	N	N	A2: stacked, total > 0
R6	Y	Y	Y	Y	N	A2: all stacked, total > 0
R7	Y	N	Y	N	Y	A3: total = 0, never negative
R8	Y	Y	Y	N	Y	A3: total = 0 or block
R9	Y	Y	Y	Y	Y	A3: total = 0 or block
R10	N	–	–	–	–	A4: reject invalid/expired code
R11	Y (same code twice)	–	–	–	–	A4: reject duplicate code

3. Optimized test set. The full combination space (price classes × promo types × tiers × number of promos) is large. Merging rules with the same action and "don't care" entries, 11 rules remain, and each is converted into one test with boundary prices (for example price = ₹1, price = discount amount, price = discount + ₹1). Add BVA on price (0, 1) and on the maximum number of promo codes allowed, and 2-way pairwise testing for any extra promo types.

4. Justification. The decision table is complete (no combination class is missed), free of redundant tests (equivalent combinations are merged), and directly exposes the negative-total

defect in rules R7–R9. The expected result for every rule is unambiguous, so defect detection is maximized with minimum test cases.

Question 9

A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.

Answer:

1. Sample structure. while (i < n) { for (j = 0; j < m; j++) { if (rec[i][j] is invalid) continue-to-next-record; if (rec[i][j] is duplicate) log; } i++; } If the "invalid" branch skips i++, the outer loop never ends.

2. Why statement coverage was insufficient. Statement coverage only checks that each line was executed at least once. A single run with a few ordinary records executes every statement, but it does not test (a) the false outcome of each condition, (b) loop termination, (c) zero, one or many iterations, or (d) the particular path (invalid record) where the counter update is bypassed. Hence 90%+ coverage can coexist with an infinite loop.

3. Cyclomatic complexity. Decision points: outer loop (1), inner loop (2), invalid check (3), duplicate check (4). V(G) = 4 + 1 = **5**, so at least 5 independent paths must be tested; V(G) > 10 would indicate the need to refactor.

Path	Description	Test input
P1	Outer loop not entered	n = 0 records
P2	Outer once, inner not entered	1 record, 0 fields
P3	Valid record, not duplicate	1 record, 1 valid field
P4	Duplicate record	2 identical records
P5	Invalid record (the path that causes the hang)	1 invalid record; test must finish within a time limit

4. Loop boundary testing (nested loops). Outer loop: 0, 1, 2, max − 1, max, max + 1 iterations. Inner loop: same set. For nested loops, set the inner loop to its minimum while testing the outer loop, then vary the inner loop with the outer at minimum, then both at typical and maximum

values. Add a timeout/watchdog assertion to every loop test so a non-terminating loop is detected as a failure, and check that the loop variable always progresses on every path.

5. Justification. Cyclomatic complexity gives the exact minimum number of tests (5) needed to cover every independent path, including the one that bypasses the loop increment. Combined with loop boundary tests, this finds the infinite-loop defect that statement coverage missed, and highlights the need to refactor the module if its complexity increases.

Question 10

A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.

Answer:

1. Evaluation of the current strategy. The existing tests probably checked each screen or input separately and followed the happy path Ordered → Preparing → Out for Delivery → Delivered. The cancellation event while in the "Out for Delivery" state was never modelled, so the system had no defined next state and the tracker froze.

2. State model. States: Ordered (S1), Preparing (S2), Out for Delivery (S3), Delivered (S4), Reassigning Driver (S5, the missing state) and Cancelled (S6). Events: restaurant accepts, food ready, driver picks up, driver delivers, driver cancels, customer cancels.

Current state	Event	Next state
Ordered	Restaurant accepts	Preparing
Ordered	Customer cancels	Cancelled
Preparing	Food ready + driver picks up	Out for Delivery
Out for Delivery	Driver delivers	Delivered
Out for Delivery	Driver cancels	Reassigning Driver (defect: no transition)
Reassigning Driver	New driver accepts	Out for Delivery
Reassigning Driver	No driver within timeout	Cancelled / refund
Delivered	Any event	No change (end state)

3. Redesigned test scenarios.

- Valid paths (0-switch): every transition once, for example Ordered → Preparing → Out for Delivery → Delivered.
- Cancellation sequences (1-switch/N-switch): Out for Delivery → Reassigning → Out for Delivery → Delivered; Out for Delivery → Reassigning → timeout → Cancelled.
- Repeated events: driver cancels twice, then a new driver accepts.
- Invalid transitions: Ordered → Delivered, Delivered → Preparing, cancel after Delivered; the system must ignore them or show an error.
- Non-functional events: app closed or network lost in each state, then reopened (the state must be restored correctly).

4. Justification. ECP and BVA test input values and ranges, but the tracker's behaviour depends on the current state and the order of events, not just on input values. The same event ("driver cancels") is valid in one state and invalid in another, and ECP/BVA cannot represent that history. State transition testing models every state-event pair, so missing transitions (such as the stuck state), invalid transitions and sequences of events are found systematically, with full state and transition coverage as the measurable criterion.

Question 11

A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.

Answer:

1. Critical assessment. Black-box testing probably verified only valid and invalid credentials, OTP expiry and IP check as separate functional rules. White-box testing probably focused on code coverage of the "happy" login path. Neither considered attacker behaviour, such as unlimited attempts, resetting counters or bypassing a factor. Security-relevant paths (lockout, rate-limiting, OTP retry) had no tests, and the two techniques were not integrated.

2. Formal path analysis (CFG of the login flow).

Path	Flow	Security question
P1	Password valid → OTP valid → IP trusted → success	Normal login works
P2	Password invalid → counter++ → error	Is the counter updated on every failure?
P3	Counter ≥ limit → account locked	Does lockout really occur and for how long?
P4	Password valid → OTP expired	Is boundary (30 s) handled correctly?
P5	Password valid → OTP invalid (retry)	Are OTP retries limited?
P6	IP unfamiliar → extra verification/block	Can the IP check be bypassed?

V(G) is computed from the decisions (password, lockout counter, OTP validity, OTP expiry, IP trust) = 5 + 1 = 6, so 6 independent paths are tested.

3. Error-guessing test ideas (based on experience and known attacks).

- Hundreds of wrong passwords from one IP, then from rotating IPs (or spoofed X-Forwarded-For header).
- Case or whitespace variations of the username to avoid the lockout counter.
- Brute-forcing a 6-digit OTP within its validity window; reusing a used OTP (replay).
- OTP request flooding; parallel (concurrent) login requests to bypass counters (race condition).
- Common/weak passwords, SQL injection and empty fields, OTP at exactly 0 and 30 seconds.

4. Hybrid strategy. (i) Use error guessing to derive attack scenarios; (ii) map each scenario to a path in the CFG and check whether the path exists and is covered; (iii) add white-box tests for lockout, rate-limit and OTP-retry branches to achieve 100% branch and path coverage of the authentication module; (iv) automate attacks using tools (for example OWASP ZAP/Burp Suite) and run them in CI; (v) re-test after every fix.

5. Justification. Error guessing brings the attacker's viewpoint and finds vulnerabilities that specifications do not mention; path analysis makes sure that the defence code is actually executed and that each decision is verified. Together they give defence in depth, systematic

coverage with creative attack tests, and a measurably stronger security posture against brute-force, replay and bypass attacks.

Question 12

An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.

Answer:

1. Evaluation of current criteria. The testing considered only normal cases (an active subscription, or one clearly expired) and ignored the date boundary and the combination of tier and date. The day of expiration is a boundary where the system must decide whether the old or the new plan applies; it was not covered, so the upgrade defect remained.

2. ECP – subscription date.

- D1: Valid, well before expiry (for example 10 days remaining).
- D2: Expiry day − 1.
- D3: **Exactly the expiry day** (boundary).
- D4: Expiry day + 1 (expired, grace period if any).
- D5: Long expired (for example 6 months).
- D6: Invalid date (null, malformed, future start date).

3. ECP – subscription type and course type. Current plan: Free, Basic, Premium. Upgrade/downgrade target: Free→Basic, Free→Premium, Basic→Premium, Basic→Basic (renewal), Premium→Basic (downgrade). Course type: Free, Basic, Premium content.

4. Structured test design.

TC	Current plan	Date state	Action	Expected
1	Basic	D1	Upgrade to Premium	Premium immediately; remaining value handled
2	Basic	D2	Upgrade to Premium	Premium active, no access loss

TC	Current plan	Date state	Action	Expected
3	Basic	D3 (expiry day)	Upgrade to Premium	Seamless upgrade; no downgrade to Free
4	Basic	D4	Upgrade to Premium	Treated as new purchase; Premium access
5	Premium	D3	Access Premium course	Access valid until end of day
6	Basic	D4	Access Basic course	Denied; reverts to Free
7	Free	n/a	Access Basic/Premium course	Denied with upgrade prompt
8	Basic	D6	Upgrade	Error message; plan unchanged
9	Basic	D3 at 23:59 / 00:00	Upgrade	Correct handling across time zone/midnight

5. Justification of completeness. Every date class and every plan type appears at least once; the boundary (expiry day ±1) is covered by BVA; tier × course type combinations check access control; and time-zone/midnight cases check the extreme edge of the boundary. As a result, upgrade, renewal, downgrade and expiry transitions are all verified, giving seamless user transitions.

Question 13

A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.

Answer:

1. Sample code. switch(mode){ case AUTO: if (lux < 30 && motion) level = 100; else if (lux < 70 || timerOn) level = 50; break; case MANUAL: level = userLevel; break; case NIGHT: level = 10; break; } (note: no default).

2. Risks of statement coverage only. Three or four tests can execute every statement, yet (a) the implicit default branch (invalid or corrupted mode value from hardware) is never taken, (b) the false outcome of if conditions is not forced, (c) short-circuit evaluation hides conditions such as motion or timerOn, and (d) wrong operators (&& vs ||) pass unnoticed. Risk: wrong brightness, flicker or an undefined output in the field.

3. Branch coverage – branches to cover. switch cases AUTO, MANUAL, NIGHT and the implicit default; first if true/false; else if true/false. Minimum tests: 6–7.

4. Condition coverage – each atomic condition true and false.

TC	mode	lux	motion	timerOn	Outcome / purpose
1	AUTO	20	T	F	lux<30 T, motion T → level 100
2	AUTO	20	F	F	motion F → goes to else-if; lux<70 T → 50
3	AUTO	50	F	F	lux<30 F; lux<70 T → 50
4	AUTO	90	F	T	lux<70 F, timerOn T → 50
5	AUTO	90	F	F	both F → level unchanged (hold)
6	MANUAL	–	–	–	level = userLevel
7	NIGHT	–	–	–	level = 10
8	Invalid (99)	–	–	–	Default: safe/last-known level, error logged

Additional tests at sensor boundaries (lux = 29, 30, 69, 70) and for out-of-range or noisy sensor values (negative, maximum, null).

5. Improved white-box strategy. (i) Add a default case; (ii) require 100% branch coverage and 100% condition (preferably MC/DC) coverage of the Boolean expressions; (iii) test using stubs/simulators for the light sensor and dimmer driver, injecting abnormal values; (iv) use a code-coverage tool on the target build; (v) review compound conditions for operator mistakes.

6. Justification. The hardware can send unexpected values (noise, disconnected sensor, glitch). Branch and condition coverage force the software to be tested against those values and against every logical combination, so the controller always ends in a defined, safe state. This reduces integration failures between sensor, software and lighting hardware.

Question 14

A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.

Answer:

1. Failure analysis. The current tests used only valid weights (positive, within range), so the input validation for zero and negative weight was never exercised. Without validation, the formula may produce zero or negative shipping cost, a division-by-zero error, or free shipping.

2. Assumed ranges. Weight: 0.1 kg to 50 kg; Zone: 1–5; Speed: Standard, Express, Overnight.

3. Robust BVA – seven values per variable: min − 1, min, min + 1, nominal, max − 1, max, max + 1 (the ones outside the range are the invalid boundaries).

Variable	Robust boundary values	Expected
Weight (kg)	−1, 0, 0.09 (below min); 0.1 (min); 0.2; 25 (nominal); 49.9; 50 (max); 50.1 (above max)	≤ 0 and out-of-range rejected; others priced correctly
Zone	0 (invalid), 1, 2, 3, 4, 5, 6 (invalid)	0 and 6 rejected
Speed	Null/blank, Standard, Express, Overnight, unknown value	Blank/unknown rejected

4. Test count. Single-fault robust BVA gives 6n + 1 = 6(3) + 1 = **19 tests** (each variable varied while the others stay nominal). Add a few worst-case combinations (for example weight 50, zone 5, Overnight for maximum cost; weight 0.1, zone 1, Standard for minimum cost).

5. Necessity of negative scenarios.

- Prevents incorrect or negative shipping charges and revenue loss.
- Avoids crashes, exceptions and divide-by-zero errors caused by bad input.
- Ensures meaningful error messages and graceful handling (robustness).
- Protects against injection attacks and corrupted data from interfaces/APIs.
- Real users and integrated systems do enter wrong values, so valid-only testing leaves a large part of the behaviour unverified.

Conclusion: Robustness BVA with negative tests ensures the logistics system rejects bad input safely and calculates correct costs for all valid inputs.

Question 15

A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.

Answer:

1. Evaluation of random testing. Random data sizes are mostly composite numbers (even or divisible by the thread count). A prime size is a special case in which the data cannot be divided equally among the worker threads, so there is always a remainder. Random testing rarely hits such special sizes, has no knowledge of the code structure, and gives no coverage measure; therefore it is unsuitable as the main technique for a mathematical module.

2. Likely defect. Chunk size = n / T (integer division). If each thread processes start = t × chunk to start + chunk, the remaining n mod T elements are skipped (or processed twice) whenever n is not divisible by the number of threads T, which is always true for a prime n > T. The result is inconsistent output.

3. CFG and Basis Path Testing. Decisions: (1) $n \leq 0$ validation, (2) thread-partition loop, (3) is it the last chunk (include the remainder?), (4) merge/reduce loop. V(G) = 4 + 1 = **5** independent paths.

Path	Description	Test data (T = 4 threads)
P1	Invalid size	n = 0 or negative → error
P2	Size smaller than thread count	n = 1, 2, 3 (prime 3)
P3	Exact division (no remainder)	n = 8, 100
P4	Remainder, last chunk takes the leftover	n = 7, 13, 101 (prime)
P5	Large dataset, merge of partial results	n = 1,00,003 (large prime)

4. Complete structured approach.

- Basis path tests P1–P5 plus loop boundary tests (0, 1, 2, T−1, T, T+1 elements).

- Equivalence classes of size: prime, composite, power of two, multiples of T, multiples of T ± 1.
- Oracle: compare the parallel result with a trusted sequential implementation, with an accepted floating-point tolerance.
- Run with different thread counts (1, 2, 4, 8) and repeat runs to check determinism.
- Supplementary concurrency tests (race detector, thread sanitizer) because path testing alone cannot detect data races.
- Measure branch and path coverage using a coverage tool.

5. Justification. Basis path testing guarantees that each independent path of the partitioning and merging logic runs at least once, and targeted prime-size tests deliberately trigger the remainder path that random testing missed. Comparison with a reference implementation verifies mathematical accuracy, while repeated multi-thread runs and boundary sizes verify stability. The result is a repeatable, measurable approach for a medical module where wrong numerical output can affect diagnosis.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand